Отчёт по лабораторной работе № 2

Первоначальная настройка git

Савкин Дмитрий Вадимович

2026-10-12

Содержание

1 Введение **5**
1.1 Актуальность 5
1.2 Цель работы 5
1.3 Задачи 5
1.4 Объект и предмет исследования 5
1.5 Методы исследования 6
1.6 Схема выполнения работы 6

2 Теоретическая часть **7**

3 Материалы и методы **8**

4 Выполнение лабораторной работы **9**

5 Результаты **15**

6 Обсуждение результатов **16**

7 Заключение **17**

Список литературы **18**

Ответы на контрольные вопросы **19**

Список иллюстраций

4.1 Устанавливаем git: `sudo dnf install git` 9
4.2 Устанавливаем GitHub CLI: `sudo dnf install gh` 9
4.3 Задаём базовые параметры git: имя, email, кодировку путей, имя ветки по умолчанию, обработку переносов строк 10
4.4 Генерируем SSH-ключ: `ssh-keygen -t ed25519`, копируем публичный ключ в буфер: `wl-copy < ~/.ssh/id_ed25519.pub` 10
4.5 Открываем раздел GitHub Settings → SSH and GPG keys для добавления ключей 11
4.6 Генерируем PGP-ключ: `gpg --full-generate-key` (RSA and RSA, 4096 бит, без ограничения срока действия) 11
4.7 Настраиваем автоматическую подпись коммитов (`user.signingkey`, `commit.gpgsign`, `gpg.program`) и авторизуемся в GitHub CLI: `gh auth login` 12
4.8 Подтверждаем активацию устройства в браузере по одноразовому коду 12
4.9 Авторизация GitHub CLI завершена успешно, вход выполнен под учётной записью Dmitry169 13
4.10 Создаём репозиторий курса на основе шаблона: `gh repo create study_2025-2026_os-intro --template=yamadharma/course-directory-student-template --public` и клонируем его: `git clone --recursive` 13
4.11 Настраиваем каталог курса: удаляем `package.json`, создаём файл `COURSE`, добавляем изменения: `git add .` 14
4.12 Сохраняем и отправляем изменения: `git commit -am 'feat(main): make course structure'`, `git push` 14

Список таблиц

1 Введение

1.1 Актуальность

Git — стандартный инструмент контроля версий в индустрии; корректная первоначальная настройка определяет корректность всей последующей истории изменений.

1.2 Цель работы

Изучить идеологию и применение средств контроля версий, освоить умения по работе с git.

1.3 Задачи

- Создать базовую конфигурацию для работы с git, создать ключи SSH и PGP, настроить подписи git, зарегистрироваться на GitHub, создать локальный каталог для выполнения заданий по предмету.

1.4 Объект и предмет исследования

Объект — система контроля версий git; предмет — параметры конфигурации пользователя и репозитория.

1.5 Методы исследования

Эмпирический метод: практическое выполнение заданий в виртуальной машине Fedora Linux с фиксацией результатов скриншотами и видеозаписью.

1.6 Схема выполнения работы

Работа выполнялась по стандартной схеме курса: выполнение действия в виртуальной машине, фиксация скриншотом, занесение в отчёт, публикация в git-репозиторий отдельным релизом.

2 Теоретическая часть

Git — распределённая система контроля версий: каждая копия репозитория содержит полную историю изменений. Перед первым использованием необходимо задать идентификацию пользователя (имя, email) и базовые настройки — без них Git не позволит создавать коммиты.

Более подробно см. в [1].

3 Материалы и методы

Используемое ПО и оборудование: хост-машина под управлением Windows, гипервизор VirtualBox, виртуальная машина с Fedora Linux (Sway), стандартные утилиты дистрибутива.

4 Выполнение лабораторной работы

Устанавливаем git: `sudo dnf install git`.

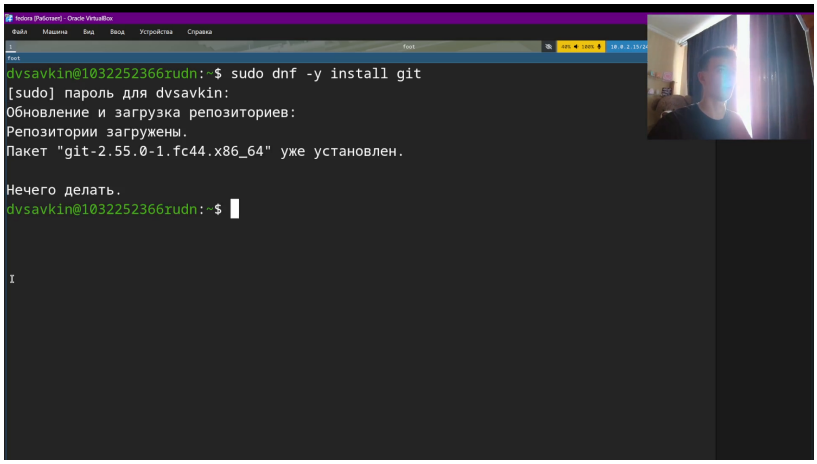


Рисунок 4.1: Устанавливаем git: `sudo dnf install git`

Устанавливаем GitHub CLI: `sudo dnf install gh`.

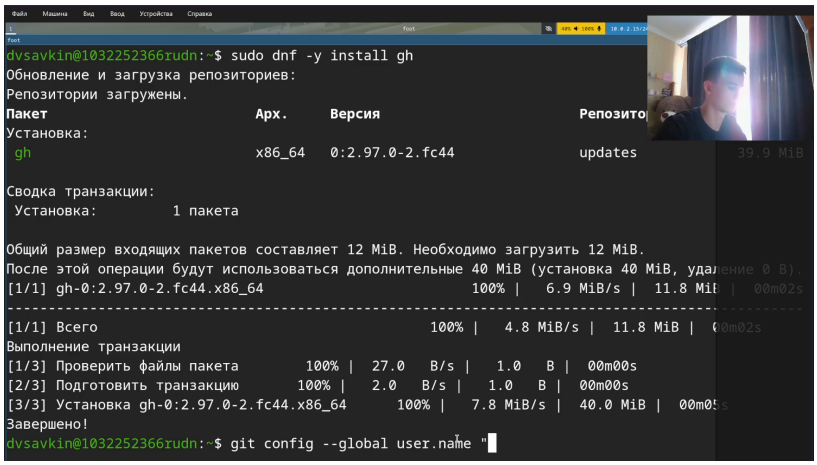


Рисунок 4.2: Устанавливаем GitHub CLI: `sudo dnf install gh`

Задаём базовые параметры git: имя, email, кодировку путей, имя ветки по умолчанию, обработку переносов строк.

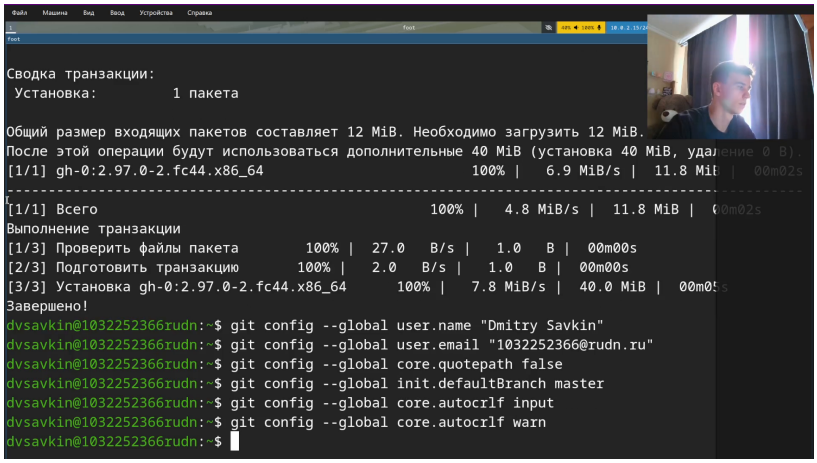


Рисунок 4.3: Задаём базовые параметры git: имя, email, кодировку путей, имя ветки по умолчанию, обработку переносов строк

Генерируем SSH-ключ: `ssh-keygen -t ed25519`, копируем публичный ключ в буфер: `wl-copy < ~/.ssh/id_ed25519.pub`.

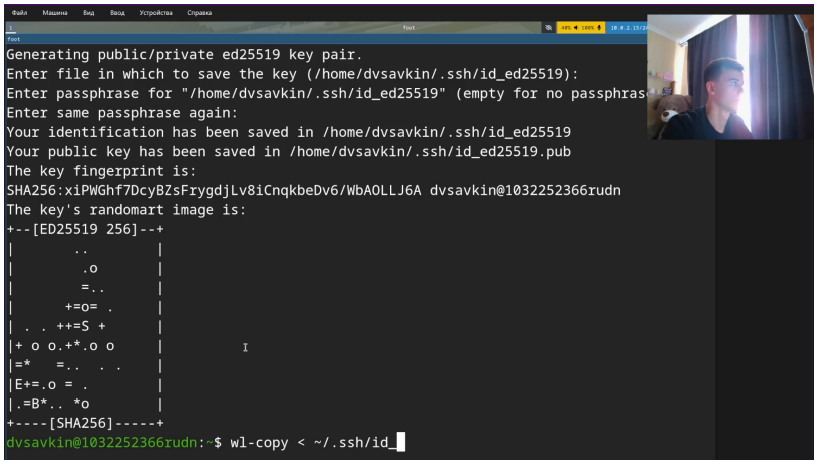


Рисунок 4.4: Генерируем SSH-ключ: `ssh-keygen -t ed25519`, копируем публичный ключ в буфер: `wl-copy < ~/.ssh/id_ed25519.pub`

Открываем раздел GitHub Settings → SSH and GPG keys для добавления ключей.

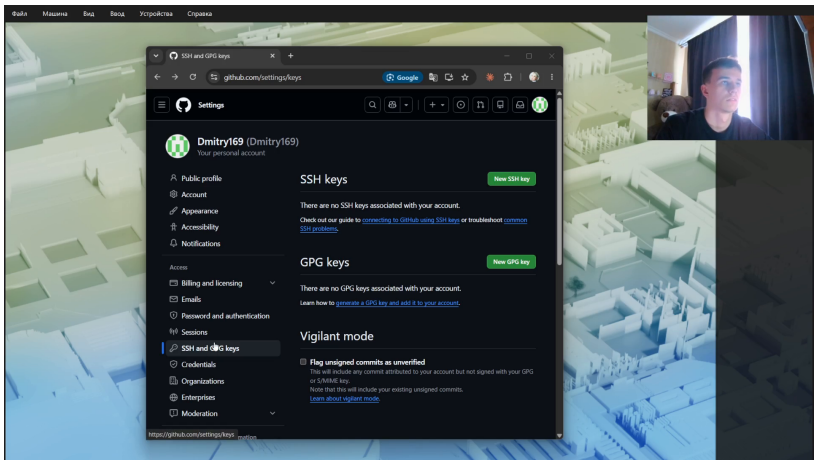


Рисунок 4.5: Открываем раздел GitHub Settings → SSH and GPG keys для добавления ключей

Генерируем PGP-ключ: `gpg --full-generate-key` (RSA and RSA, 4096 бит, без ограничения срока действия).

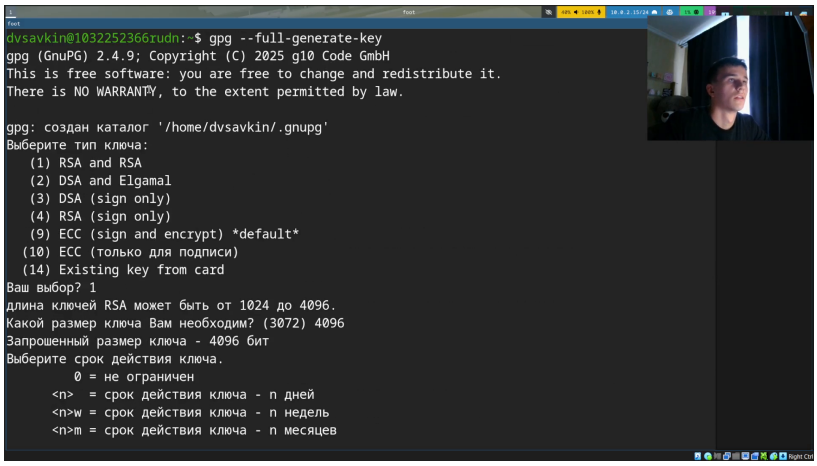


Рисунок 4.6: Генерируем PGP-ключ: `gpg --full-generate-key` (RSA and RSA, 4096 бит, без ограничения срока действия)

Настраиваем автоматическую подпись коммитов (`user.signingkey`, `commit.gpgsign`, `gpg.program`) и авторизуемся в GitHub CLI: `gh auth login`.

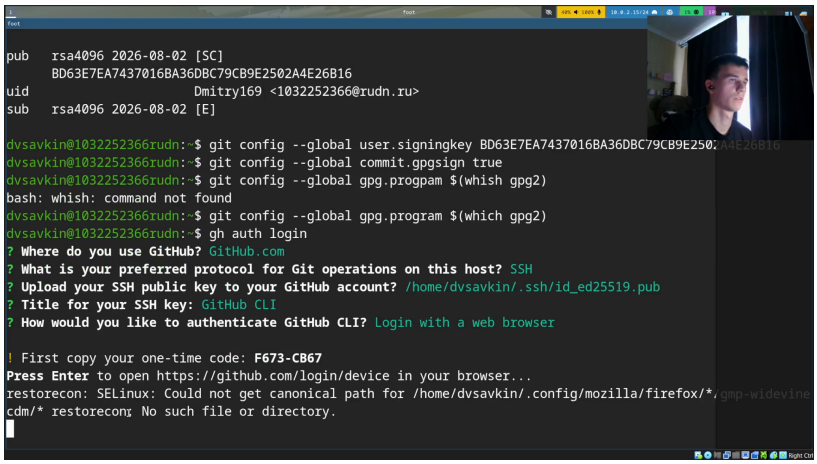


Рисунок 4.7: Настраиваем автоматическую подпись коммитов (`user.signingkey`, `commit.gpgsign`, `gpg.program`) и авторизуемся в GitHub CLI: `gh auth login`

Подтверждаем активацию устройства в браузере по одноразовому коду.

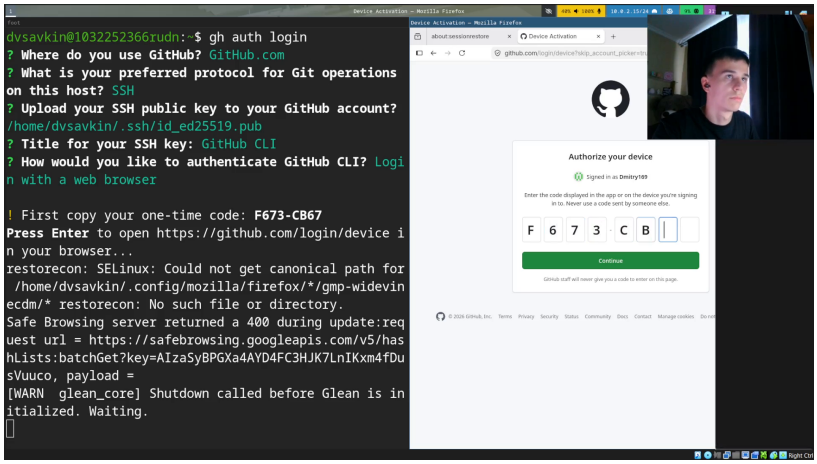


Рисунок 4.8: Подтверждаем активацию устройства в браузере по одноразовому коду

Авторизация GitHub CLI завершена успешно, вход выполнен под учётной записью Dmitry169.

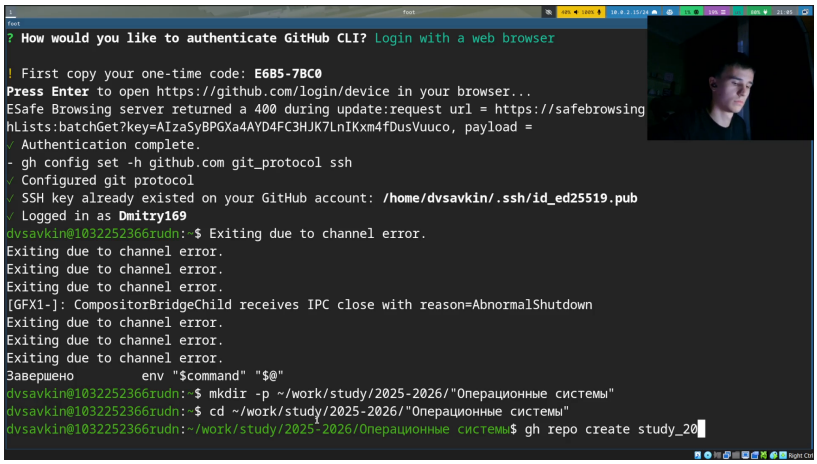


Рисунок 4.9: Авторизация GitHub CLI завершена успешно, вход выполнен под учётной записью Dmitry169

Создаём репозиторий курса на основе шаблона: `gh repo create study_2025-2026_os-intro --template=yamadharma/course-directory-student-template --public` и клонируем его: `git clone --recursive`.

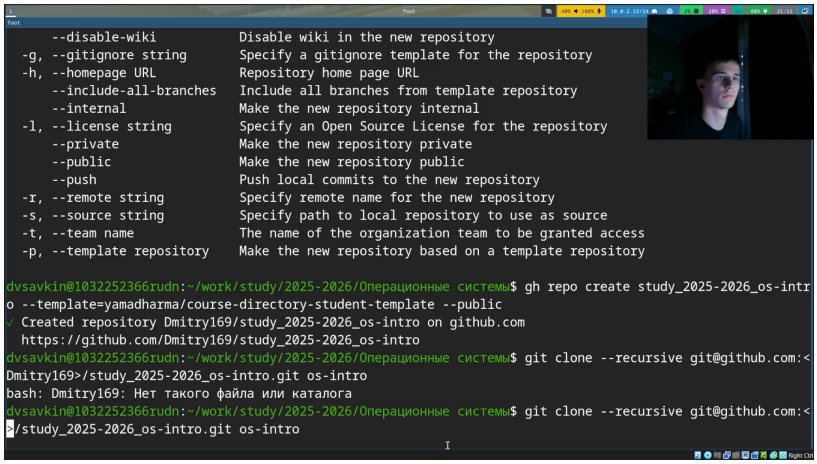


Рисунок 4.10: Создаём репозиторий курса на основе шаблона: `gh repo create study_2025-2026_os-intro --template=yamadharma/course-directory-student-template --public` и клонируем его: `git clone --recursive`

Настраиваем каталог курса: удаляем `package.json`, создаём файл `COURSE`, добавляем изменения: `git add .`.

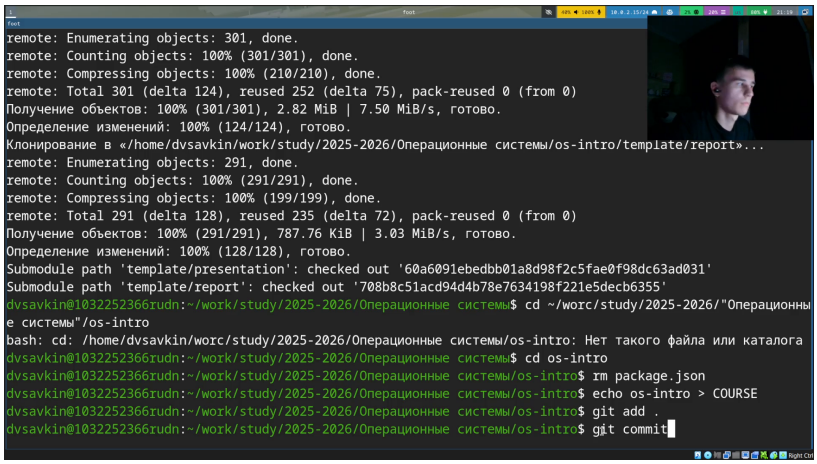


Рисунок 4.11: Настраиваем каталог курса: удаляем `package.json`, создаём файл `COURSE`, добавляем изменения: `git add .`

Сохраняем и отправляем изменения: `git commit -am 'feat(main): make course structure'`, `git push`.

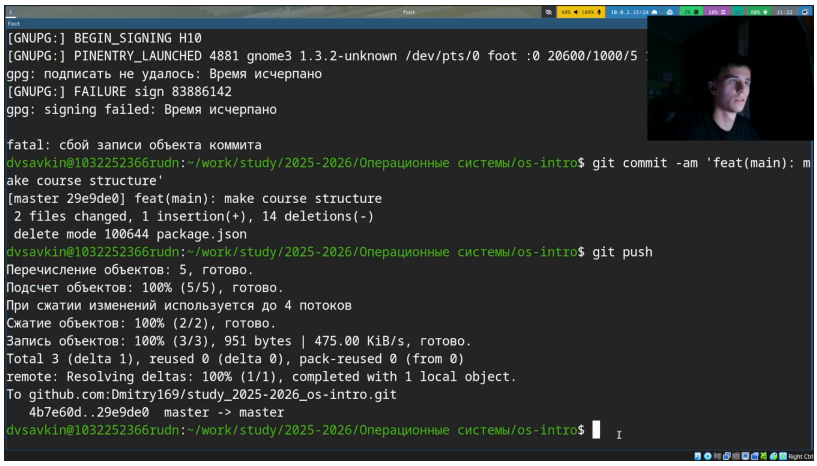


Рисунок 4.12: Сохраняем и отправляем изменения: `git commit -am 'feat(main): make course structure'`, `git push`

5 Результаты

В ходе выполнения работы получены следующие объективные результаты: Изучены принципы работы распределённой системы контроля версий git, получены практические навыки настройки git, генерации SSH- и PGP-ключей, подписи коммитов и работы с GitHub через терминал, в том числе через утилиту gh. Создан и настроен рабочий репозиторий курса на основе шаблона, в который отправлен подписанный коммит.

6 Обсуждение результатов

Результат соответствует теоретическим ожиданиям: без первоначальной настройки git отказывается создавать коммиты, что подтверждает необходимость этого шага.

7 Заключение

Изучены принципы работы распределённой системы контроля версий git, получены практические навыки настройки git, генерации SSH- и PGP-ключей, подписи коммитов и работы с GitHub через терминал, в том числе через утилиту gh. Создан и настроен рабочий репозиторий курса на основе шаблона, в который отправлен подписанный коммит.

Список литературы

1. *Software Freedom Conservancy*. Pro Git.. — 2014.

Ответы на контрольные вопросы

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются? Программные средства для фиксации изменений в файлах проекта при совместной работе нескольких человек: сохранение истории, слияние правок, откат к более ранним версиям.

2. Объясните понятия VCS: хранилище, commit, история, рабочая копия. Хранилище — репозиторий со всеми версиями проекта; commit — зафиксированный набор изменений; история — последовательность коммитов; рабочая копия — локальная версия файлов пользователя.

3. Централизованные и децентрализованные VCS? Централизованные (SVN) — один сервер. Децентрализованные (git) — полная копия у каждого.

4. Работа с VCS в одиночку. `init`, правки, `status`/`diff`, `add`, `commit`.

5. Работа с общим хранилищем. `pull`, ветка, правки, `add`, `commit`, `push`.

6. Задачи git. Версионирование, совместная работа, слияние, ветки, бэкап.

7. Основные команды git. `init`, `status`, `diff`, `add`, `rm`, `commit`, `pull`, `push`, `checkout`, `merge`, `branch`.

8. Локально и с удалённым репозиторием. Локально: `init`, `add`, `commit`. Удалённо: `clone`, `pull`, `push`.

9. Ветви (branches)? Параллельная линия разработки для экспериментов без риска для основной ветки.

10. Игнорирование файлов? Через `.gitignore` — чтобы служебные файлы не попадали в отслеживание.